SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECHNOLOGY, SURAT

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CS103: DESIGN AND ANALYSIS OF ALGORITHMS

M.Tech. (DS) Ird Year

Lab Assignment : Backtracking Algorithms

Experiment 1: Constrained 8-Queens Problem

Problem statement

Design a backtracking algorithm to solve the 8-Queens problem on an \\(8 \times 8\\) chessboard. No two queens may share a row, column, or diagonal. Unlike the conventional problem, your algorithm must also support forbidden positions, pre-placed queens, and additional search statistics.
Question 1.1 — Develop a program that accepts the following as input:

- A set of forbidden cells where queens cannot be placed.

- A set of pre-placed queens whose positions must remain fixed.

- The requirement that exactly eight queens must be placed on the board.

Your program must:

1. Determine whether a valid arrangement exists.

2. Print all valid arrangements, not just the first one.

3. Count the total number of valid arrangements.

4. Reject contradictory input, such as two pre-placed queens attacking each other or a pre-placed queen occupying a forbidden cell.

5. Prune a branch immediately when a newly placed queen attacks a previously placed queen.

6. Avoid exploring symmetric arrangements as duplicates where the imposed constraints permit symmetry reduction.

Experiment 2: Sum of Subsets Problem

Problem statement

Given a set of positive integers and a target sum, find all subsets whose elements sum exactly to the target. Extend the classical backtracking algorithm to handle cardinality constraints, mandatory and excluded elements, and optimization over feasible subsets.

Question 2.1 — Constrained sum of subsets

Write a backtracking program that accepts:

- A set of n positive integers.

- A target sum T.

- A minimum and maximum permitted subset size, \\(k_{\min}\\) and \\(k_{\max}\\).

- A set of mandatory elements that must appear in every valid subset.

- A set of excluded elements that cannot appear in any valid subset.

Your algorithm must:

1. Enumerate all subsets satisfying the target sum and every additional constraint.

2. Print each valid subset and count the total number of valid subsets.

3. Handle duplicate values correctly by distinguishing between element positions, even when two elements have the same numerical value.

4. Reject invalid input, including overlapping mandatory and excluded element indices.

5. Prune a branch when its current sum exceeds T.

6. Prune a branch when the maximum possible sum of its remaining eligible elements cannot reach T.

7. Use cardinality-based pruning to stop branches that cannot satisfy the required subset size.

Test your program on the following input:

S={3,5,6,7,9,11,12}, T=21

Use the following constraints:

- Minimum subset size: 2

- Maximum subset size: 4

- Mandatory element: 3

- Excluded element: 7

Here, mandatory and excluded elements refer to the numerical values in the set.

Report every valid subset and explain why each rejected branch was pruned.

Experiment 3: Graph Coloring Using Backtracking

Problem statement

Given an undirected graph (G=(V,E)), assign colors to its vertices so that adjacent vertices have different colors. Extend the standard graph-coloring algorithm to support pre-colored vertices, restricted color lists, and minimum-color optimization.

Question 3.1 — List-constrained graph coloring

Design a backtracking algorithm that accepts:

- An undirected graph represented by an adjacency list or adjacency matrix.

- A maximum of \\(m\\) available colors.

- A list of allowed colors for each vertex.

- A set of pre-colored vertices.

Your algorithm must:

1. Determine whether a valid coloring exists.
2. Find and print all valid colorings using the permitted colors.
3. Reject any assignment in which adjacent vertices share a color.
4. Ensure that each pre-colored vertex retains its original color.
5. Ensure that each vertex receives a color from its allowed list.
6. Prune a branch as soon as a color violates an adjacency or list constraint.
7. Report the number of recursive calls and branches pruned.